do that there

Look at something on your Mac, say what should happen — “archive that”, “put this there” — and an agent does it inside the apps you already use.

`hold ⌥ space  ·  “put this there”  ·  release`

Omar Contreras Cabanillas · MIT Sloan MBA ’27

October 29, 2026

Hands-free computer control today is slow and unreliable

1 in 10

US adults has some difficulty using their hands and fingers (NHIS, Washington Group)

1 in 50

genuinely struggles — repetitive strain injuries (RSI), tremors, paralysis, other motor impairments

2018

Dragon (former voice assistive tool) discontinued its Mac version; leading eye trackers don't officially support macOS

Operating a Mac by voice today

- With **voice/grid mechanisms,** to click one button: say "show numbers", read the number the screen paints on it, **speak the number — hundreds of times a day**
- **Anything more takes command languages or user-written scripts;** many apps don't respond properly, so people get stuck and ask for help
- 2026's AI agents can act on a screen, but none knows where your attention is — **"this" and "there" cost a paragraph of explanation**

Who this serves

- **First:** programmers with RSI and daily voice-control users, who already pay monthly for tools that make them script
- **Next:** people with tremors and motor disabilities, through assistive-technology funders
- **Eventually:** anyone whose hands are busy

A macOS agent that does what you mean where you mean it

01

Hold a key, speak, release

⌥ space · “put this there” — no wake word, no command language

02

Each word is transcribed and timestamped

On-device transcription returns each word with the moment it was said

03

Attention Resolver

Binds “this” and “there” to screen elements: live selection and focus, the pointer at that word’s instant, and what the last turn acted on

04

One planned action

An LLM planner turns the spoken command plus resolved candidates into exactly one tool call

05

Act and verify

Acts through the Accessibility API, re-checks the screen, and verifies the desired change of state

What a user will be able to do

- **Everyday actions in real apps** — move, open, rename, archive, reply, run — starting with Finder, Mail, and Terminal, with no grid and no scripts in the common case
- **Stay in control** — numbers are painted on screen only when the reference is genuinely ambiguous; risky changes show a warning and wait for an explicit “yes”; irreversible ones require an explicit confirm
- **Interrupt anytime** — undo on a word; any mouse or keyboard input aborts the agent instantly; the user’s own input is never taken over

Architecture, technologies, and progress so far

Swift shell — input and screen access

Menu-bar app: talk key, pointer sampler, input timestamps, on-device speech, Accessibility API actions, number overlay

`JSON-RPC 2.0 · localhost WebSocket · per-launch token`

Python brain — resolution and planning

Attention resolver, planner adapters, confirmation tiers, referent log, local traces, metrics

Technologies and external systems

- **Speech:** Apple SpeechAnalyzer — on-device, per-word audio time ranges (WWDC25)
- **Acting:** macOS Accessibility API (AXUIElement) — snapshot, act, re-snapshot to verify; fall back to event click, then keystroke
- **Planning:** Claude Sonnet 5.5 — one tool call per turn, versioned prompts
- **Evaluation:** a built-in baseline mode runs the same utterance the incumbent way (screenshot + cursor only), so every claim is measured
- **Prior work:** Bolt's Put-That-There (MIT, 1980); a 2026 MIT Media Lab × Microsoft Research study — 31% fewer words when the assistant knows where you look (n = 36)

Completed so far

Specification, architecture, and a working repository with CI started a couple of weeks ago · an accessibility write test built and run on 7 Mac apps · Finder, Mail, and Terminal shown to be reliably drivable · 130 Swift tests and a Python test suite passing

What I can realistically complete, and what could prevent it

In scope this semester

- Attention from pointer, live selection, and referent history — signals that are reliable today
- Three demo apps proven drivable: Finder, Mail, Terminal
- ~20 command types end to end, with confirmation tiers, undo, and instant abort
- Baseline mode and local traces, so results are measured, not asserted

Deliberately out of scope

- Webcam gaze tracking — a later extension for people who cannot point (planned for spring)
- Browsers and Electron apps — they failed an accessibility write test performed in October
- Windows, distribution, and App Store polish

What could prevent success, and the plan for each

Risk	Mitigation
Per-word speech timing may be coarse or late on device	Measure it first, in the opening build session; fall back to utterance-level timing plus pointer dwell
Accessibility actions fail silently in some apps	Verify every action by tree diff; fall back to event click, then keystroke; demo apps already chosen on test evidence
Planner latency or cost makes turns feel slow	Small ranked candidate lists; a rule-based fast path for the common verbs; latency is a tracked metric
Solo project, scope creep	Go/no-go checkpoints with written pass criteria; demo apps fixed at three; if time runs short, reduce the number of commands rather than verification or safety

Milestones and measurable success criteria

Oct 29
Proposal
This deck; instructor feedback on direction

Nov 2
Sensing layer done
The app logs a complete turn: timed words, pointer, machine state

Nov 9
First command works
Resolver tests passing; first spoken command performed end to end

Mid-Nov
Midpoint check-in
Live demo: commands acted in Finder, Mail, Terminal

Nov 23
Feature complete
Confirmation tiers, undo, and baseline mode working

Early Dec
Final
Evaluation run, README, polish, final presentation

How I will demonstrate that it works

- **Resolution accuracy:** on a scripted set of ~40 utterances across the three apps, the resolver picks the intended element in ≥ 80% of turns (target)
- **Fewer words than the status quo:** the same tasks completed with measurably fewer spoken words than the built-in baseline mode (screenshot + cursor, no attention signals)
- **Responsive:** median time from key release to a verified action of a few seconds, measured from the logged traces
- **Safe:** zero irreversible actions without explicit confirmation; any mouse or key input aborts — demonstrated live